

Automated File Backup, Encryption, and Cloud Sync Using S3, Bash, GPG, and Terraform

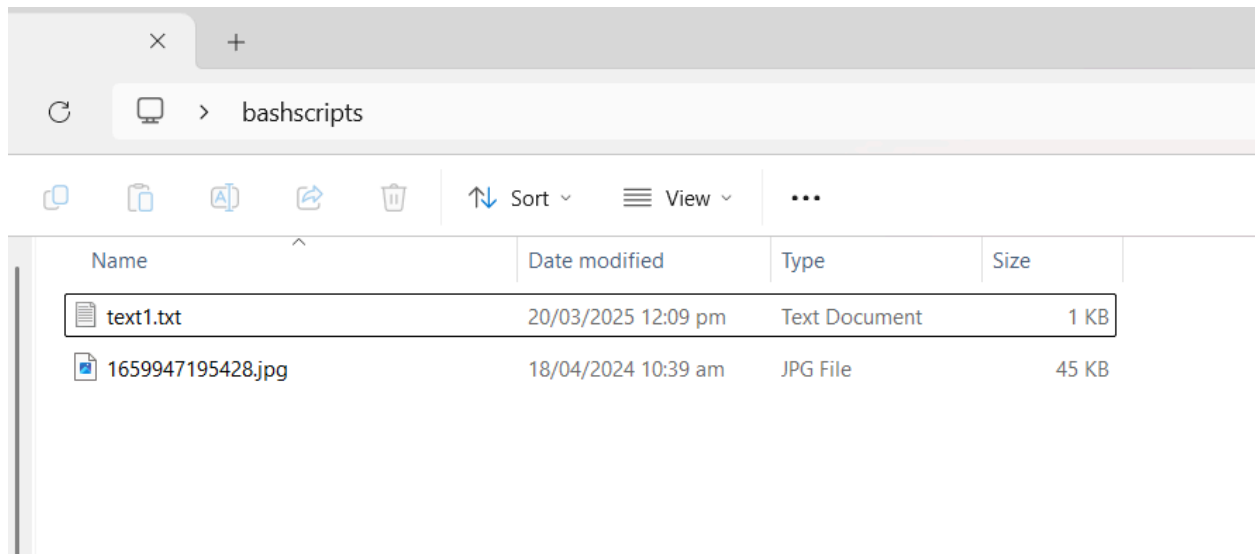
This project is a Linux-based automation project that automatically creates backups of files in a directory, encrypts them using GPG, and uploads them to AWS S3. The project integrates Bash scripting for automation, GPG for encryption, AWS CLI for cloud storage, and Terraform for infrastructure as code.

Tech Stack: Bash, GPG, AWS S3, Terraform, Cron Jobs

Step 1: Setting Up the Backup Script

The first step was to create a Bash script (myscript2.sh) that automates the backup process. It copies files from a specified source directory to a backup directory and compresses them into a .tar.gz archive with a timestamp.

Source directory:



Backup Directory:

> backups >			
Sort View			
Name	Date modified	Type	Size
backup_2025-03-20 14· 25· 52.tar.gz	20/03/2025 2:25 pm	Compressed Archi...	45 KB
backup_2025-03-20 14· 27· 25.tar.gz	20/03/2025 2:27 pm	Compressed Archi...	45 KB
backup_2025-03-20 14· 28· 43.tar.gz	20/03/2025 2:28 pm	Compressed Archi...	45 KB
backup_2025-03-20 20· 52· 31.tar.gz.gpg	20/03/2025 8:52 pm	GPG File	45 KB
decrypted_backup.tar.gz	20/03/2025 2:55 pm	Compressed Archi...	45 KB

> backups > backup_2025-03-20 14· 25· 52.tar.gz			
Sort View			
Name	Type	Size	Date modified
1659947195428.jpg	JPG File	45 KB	18/04/2024 10:39 am
text1.txt	Text Document	1 KB	20/03/2025 12:09 pm

Logs were also created to keep track of the results each time the bash script was run.

> logs			
Sort View			
Name	Date modified	Type	Size
backup.log	20/03/2025 8:52 pm	Text Document	3 KB

```
File Edit View
2025-03-20 13:25:27 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 13:48:40 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 13:53:19 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:00:35 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:00:39 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:00:41 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:00:42 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:00:53 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:04:07 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:10:09 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:15:46 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:22:18 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:22:54 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:25:25 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:25:52 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:27:25 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:28:43 - Deleted old backup: /mnt/c/users/Miguel Huerto/Desktop/backups/backup_2025-03-20
14:22:18.tar.gz
2025-03-20 14:28:43 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 14:46:04 - Deleted old backup: /mnt/c/users/Miguel Huerto/Desktop/backups/backup_2025-03-20
14:22:54.tar.gz
2025-03-20 14:46:05 - Encrypted backup and deleted unencrypted file: /mnt/c/users/Miguel
Huerto/Desktop/backups/backup_2025-03-20 14:46:04.tar.gz.gpg
2025-03-20 14:46:05 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 20:50:23 - Deleted old backup: /mnt/c/users/Miguel Huerto/Desktop/backups/backup_2025-03-20
14:25:25.tar.gz
2025-03-20 20:50:24 - Encrypted backup and deleted unencrypted file: /mnt/c/users/Miguel
Huerto/Desktop/backups/backup_2025-03-20 20:50:23.tar.gz.gpg
2025-03-20 20:50:24 - Failed to upload backup to S3: /mnt/c/users/Miguel
Huerto/Desktop/backups/backup_2025-03-20 20:50:23.tar.gz.gpg
2025-03-20 20:50:24 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
2025-03-20 20:52:32 - Encrypted backup and deleted unencrypted file: /mnt/c/users/Miguel
Huerto/Desktop/backups/backup_2025-03-20 20:52:31.tar.gz.gpg
2025-03-20 20:52:35 - Backup uploaded to S3: /mnt/c/users/Miguel Huerto/Desktop/backups/backup_
2025-03-20 20:52:31.tar.gz.gpg
2025-03-20 20:52:35 - Backup created at /mnt/c/users/Miguel Huerto/Desktop/backups
```

Step 2: Implementing Automated Backup Rotation

To avoid unnecessary storage consumption, the script automatically deletes older backups, keeping only the most recent five. It lists all backups in the directory, sorts them by creation time, and removes the oldest ones beyond the limit. This ensures that the backup folder doesn't get overcrowded over time.

backups			
Sort View ...			
Name	Date modified	Type	Size
backup_2025-03-20 14: 25: 52.tar.gz	20/03/2025 2:25 pm	Compressed Archi...	45 KB
backup_2025-03-20 14: 27: 25.tar.gz	20/03/2025 2:27 pm	Compressed Archi...	45 KB
backup_2025-03-20 14: 28: 43.tar.gz	20/03/2025 2:28 pm	Compressed Archi...	45 KB
backup_2025-03-20 20: 52: 31.tar.gz.gpg	20/03/2025 8:52 pm	GPG File	45 KB
decrypted_backup.tar.gz	20/03/2025 2:55 pm	Compressed Archi...	45 KB

Step 3: Encrypting the Backups Using GPG

For security, the backups are encrypted using **GPG with AES-256 encryption**. After creating the .tar.gz file, the script encrypts it and deletes the original unencrypted version. This ensures that even if someone gains access to the backup files, they won't be able to read the contents without the passphrase.

backups			
Sort View ...			
Name	Date modified	Type	Size
backup_2025-03-20 14: 25: 52.tar.gz	20/03/2025 2:25 pm	Compressed Archi...	45 KB
backup_2025-03-20 14: 27: 25.tar.gz	20/03/2025 2:27 pm	Compressed Archi...	45 KB
backup_2025-03-20 14: 28: 43.tar.gz	20/03/2025 2:28 pm	Compressed Archi...	45 KB
backup_2025-03-20 20: 52: 31.tar.gz.gpg	20/03/2025 8:52 pm	GPG File	45 KB
decrypted_backup.tar.gz	20/03/2025 2:55 pm	Compressed Archi...	45 KB

Step 4: Uploading Backups to AWS S3

Once the backup is encrypted, it is uploaded to an AWS S3 bucket using the AWS CLI. The script verifies if the upload was successful and logs the result. This provides off-site storage, preventing data loss in case of local system failures.

General purpose buckets (5) [Info](#) All AWS Regions

Find buckets by name

< 1 >

	Name	AWS Region	IAM Access Analyzer	Creation date
☐	cf-templates-5ymzdaluv67n-ap-southeast-2	Asia Pacific (Sydney) ap-southeast-2	View analyzer for ap-southeast-2	February 7, 2025, 14:25:49 (UTC+08:00)
☐	cloud-resume-proj-huerto0	Asia Pacific (Sydney) ap-southeast-2	View analyzer for ap-southeast-2	March 6, 2025, 16:49:51 (UTC+08:00)
☐	elasticbeanstalk-ap-southeast-2-084828560751	Asia Pacific (Sydney) ap-southeast-2	View analyzer for ap-southeast-2	February 26, 2025, 19:00:35 (UTC+08:00)
☐	terraform-bucket-huerto2025	US East (N. Virginia) us-east-1	View analyzer for us-east-1	March 20, 2025, 20:46:55 (UTC+08:00)
☐	terraformprojecthuerto	Asia Pacific (Sydney) ap-southeast-2	View analyzer for ap-southeast-2	February 25, 2025, 15:40:40 (UTC+08:00)

Objects (2)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

Show versions

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	backup_2025-03-20 20:52:31.tar.gz.gpg	gpg	March 20, 2025, 20:52:35 (UTC+08:00)	45.0 KB	Standard
☐	test.txt	txt	March 20, 2025, 20:52:04 (UTC+08:00)	10.0 B	Standard

backup_2025-03-20 20:52:31.tar.gz.gpg [Info](#)

Copy S3 URI

Download

Open

Object actions

PropertiesPermissionsVersions

Object overview

Owner

miguelhhuerto+nwaws

AWS Region

US East (N. Virginia) us-east-1

Last modified

March 20, 2025, 20:52:35 (UTC+08:00)

Size

45.0 KB

Type

gpg

Key

[backup_2025-03-20 20:52:31.tar.gz.gpg](#)

S3 URI

[s3://terraform-bucket-huerto2025/backup_2025-03-20 20:52:31.tar.gz.gpg](#)

Amazon Resource Name (ARN)

[arn:aws:s3:::terraform-bucket-huerto2025/backup_2025-03-20 20:52:31.tar.gz.gpg](#)

Entity tag (Etag)

[Ocaba1d0263344b64e1864fb06786b55](#)

Object URL

[https://terraform-bucket-huerto2025.s3.us-east-1.amazonaws.com/backup_2025-03-20+20%3A52%3A31.tar.gz.gpg](#)

Step 5: Automating the Process with Cron Jobs

To make the backup process fully automated, a **cron job** is set up to execute the script every day at **2:00 PM**.

```
GNU nano 6.2 /tmp
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 14 * * * /mnt/c/users/Miguel\ Huerto/Desktop/myscript2.sh
```

Step 6: Automating S3 Bucket Creation with Terraform

Instead of manually creating an S3 bucket, Terraform is used to define and provision the cloud infrastructure. A main.tf file is written to configure the S3 bucket, and Terraform is initialized and applied to ensure the bucket exists before the script runs. This makes the setup reproducible and prevents errors due to missing storage.

```

GNU nano 6.2
provider "aws" {
  region = "us-east-1" # Change to your preferred region
}

resource "aws_s3_bucket" "storage" {
  bucket = "terraform-bucket-huerto2025"
}

resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.storage.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "lifecycle" {
  bucket = aws_s3_bucket.storage.id

  rule {
    id      = "delete-old-backups"
    status  = "Enabled"

    expiration {
      days = 30 # Auto-delete backups older than 30 days
    }
  }
}

```

Terraform will perform the following actions:

```

# aws_s3_bucket.storage will be created
+ resource "aws_s3_bucket" "storage" {
+   acceleration_status = (known after apply)
+   acl                 = (known after apply)
+   arn                 = (known after apply)
+   bucket              = "terraform-bucket-huerto2025"
+   bucket_domain_name  = (known after apply)
+   bucket_prefix       = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy       = false
+   hosted_zone_id      = (known after apply)
+   id                  = (known after apply)
+   object_lock_enabled = (known after apply)
+   policy              = (known after apply)
+   region              = (known after apply)
+   request_payer       = (known after apply)
+   tags_all            = (known after apply)
+   website_domain       = (known after apply)
+   website_endpoint    = (known after apply)
+
+   cors_rule (known after apply)
+   grant (known after apply)
+   lifecycle_rule (known after apply)
+   logging (known after apply)
+   object_lock_configuration (known after apply)
+   replication_configuration (known after apply)
+   server_side_encryption_configuration (known after apply)
+   versioning (known after apply)
+   website (known after apply)
}

```

Final Outcome

The result is a fully automated, secure backup system that encrypts and stores data in the cloud. It manages storage efficiently by deleting old backups and runs on a fixed schedule without manual input. This project uses Linux automation, encryption, AWS cloud storage, and Infrastructure as Code (IaC).

```
root@DESKTOP-JOJG6NP:/mnt/c/users/Miguel Huerto/Desktop# bash myscript2.sh
upload: backups/backup_2025-03-20 20:52:31.tar.gz.gpg to s3://terraform-bucket-huerto2025/backup_2025-03-20 20:52:31.tar.gz.gpg
Backup completed! Archive saved as /mnt/c/users/Miguel Huerto/Desktop/backups/backup_2025-03-20 20:52:31.tar.gz.gpg and uploaded to S3.
root@DESKTOP-JOJG6NP:/mnt/c/users/Miguel Huerto/Desktop#
```

Final bash script:

```
#!/bin/bash

source_dir="/mnt/c/users/Miguel Huerto/Desktop/bashscripts"
backup_dir="/mnt/c/users/Miguel Huerto/Desktop/backups"
log_file="/mnt/c/users/Miguel Huerto/Desktop/logs/backup.log"
max_backups=5
s3_bucket="terraform-bucket-huerto2025"

cp -r "$source_dir"/* "$backup_dir" 2>> "$log_file"

timestamp=$(date '+%Y-%m-%d %H:%M:%S')
backup_archive="$backup_dir/backup_${timestamp}.tar.gz"

tar -czf "$backup_archive" -C "$source_dir" .
find "$backup_dir" ! -name "*.tar.gz" -type f -delete

ls -t /mnt/c/users/Miguel Huerto/Desktop/backups/*.tar.gz | tail -n +6 | while read -r file; do
    rm -f "$file"
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Deleted old backup: $file" >> "$log_file"
done

gpg --batch --passphrase "YourSecurePassword" --symmetric --cipher-algo AES256 "$backup_archive"
if [ -f "$backup_archive.gpg" ]; then
    rm -f "$backup_archive"
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Encrypted backup and deleted unencrypted file: $backup_archive.gpg" >> "$log_file"

    aws s3 cp "$backup_archive.gpg" "s3://$s3_bucket/"
    if [ $? -eq 0 ]; then
        echo "$(date '+%Y-%m-%d %H:%M:%S') - Backup uploaded to S3: $backup_archive.gpg" >> "$log_file"
    else
        echo "$(date '+%Y-%m-%d %H:%M:%S') - Failed to upload backup to S3: $backup_archive.gpg" >> "$log_file"
    fi
else
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Encryption failed for: $backup_archive" >> "$log_file"
fi
```